

دانشگاه صنعتی شریف



دانشکده مهندسی برق

درس سیستم‌های نهفته

تمرین چهارم

طراحی و پیاده‌سازی سیستم پایگاه داده توزیع شده

استاد درس:

دکتر ایمان غلامپور

طراحان تمرین:

علی حبیبی

حسین بشارتی

نیم‌سال دوم تحصیلی:

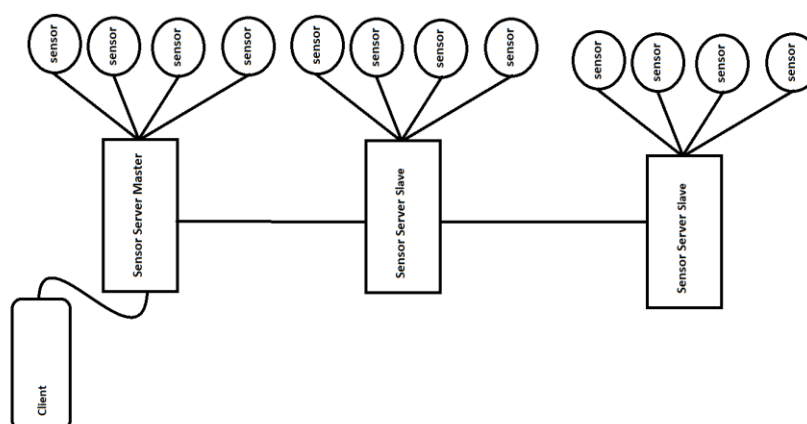
۱۴۰۴-۱۴۰۵

شرح کلی تمرین

شما به عنوان مهندس یک سیستم مانیتورینگ، مسئول طراحی و پیاده سازی سامانه ای برای مدیریت و جمع آوری داده های سنسورهای یک هتل هستید. در این سناریو، هر طبقه هتل دارای چندین سنسور مانند سنسور دما، رطوبت، حرکت و سایر سنسورهای محیطی است.

هدف این تمرین، طراحی یک سیستم پایگاه داده توزیع شده است که در آن:

- هر نود، داده های سنسورهای متصل به خود را دریافت و نگهداری کند.
- یک نود به عنوان Master وجود داشته باشد که درخواست های اپراتور را دریافت کرده و پاسخ نهایی را ارائه دهد.
- نودهای Slave فقط با نود Master ارتباط داشته باشند و مستقیماً به اپراتور پاسخ ندهند.
- اگر داده موردنظر در یک نود موجود نباشد، درخواست به نود بعدی ارسال شود تا در نهایت پاسخ به Master برگردد و Master نتیجه را به اپراتور اعلام کند.



قوانین عمومی تمرین

۱. در این تمرین فقط مجاز به استفاده از زبان های برنامه نویسی C یا C++ هستید.
۲. این تمرین شامل شش بخش است. برای هر بخش، یک پوشه جداگانه با شماره همان بخش ایجاد کنید. کدها، اسکریپت ها، پاسخ ها، فایل راهنما و گزارش مربوط به هر بخش باید داخل همان پوشه قرار گیرد.
۳. در گزارش هر بخش، از توضیح خط به خط کد خودداری کنید. گزارش باید از دیدگاه طراح سیستم نوشته شود و معماری، اجزاء، نحوه ارتباط بین اجزاء، مسیر درخواست و پاسخ، و تصمیم های طراحی سیستم را توضیح دهد.
۴. در هر گزارش، لازم است حداقل یک دیاگرام معماری سیستم یا دیاگرام جریان درخواست و پاسخ قرار داده شود.

۵. در هر بخش، حتماً یک فایل Markdown با پسوند md، قرار دهید و در آن نحوه کامپایل، اجرای برنامه، ارسال درخواست، وابستگی‌های موردنیاز و روش تست برنامه را توضیح دهید.
۶. برای کامپایل کردن کدها در هر بخش، حتماً باید از Makefile استفاده شود.
۷. در هر بخش، یک اسکریپت Bash قرار دهید که کدها را کامپایل و اجرا کند. در صورت نبود این اسکریپت، آن بخش تصحیح نخواهد شد و نمره آن بخش صفر در نظر گرفته می‌شود.
۸. برای توسعه بهتر کدها، پیشنهاد می‌شود از Git استفاده کنید تا بتوانید نسخه‌های مختلف کد خود را مدیریت کنید. استفاده از Git اجباری نیست، اما شامل امتیاز خواهد بود.
۹. برای مقداردهی اولیه پایگاه داده، از فایل‌هایی که در اختیار شما قرار داده شده است استفاده کنید. توجه داشته باشید که این فایل‌ها در زمان تست تغییر خواهند کرد؛ بنابراین از قراردادن مقادیر ثابت و پاسخ‌های آماده در کد خودداری کنید. در صورت مشاهده این مورد، کل نمره تمرین صفر خواهد شد.
۱۰. آدرس IP، شماره پورت، مسیر فایل پایگاه داده و سایر تنظیمات اجرایی نباید به صورت مقدار ثابت در کد نوشته شوند. این موارد باید از طریق فایل پیکربندی، آرگومان خط فرمان یا متغیرهای محیطی دریافت شوند.
۱۱. استفاده از ابزارهای هوش مصنوعی در این تمرین آزاد است. معیار اصلی ارزیابی، کیفیت طراحی سیستم، درستی پیاده‌سازی، قابلیت اجرا، ساختار پروژه و تحلیل ارائه‌شده در گزارش خواهد بود.

الزام تهیه ویدئوی تست نهایی

پس از اتمام پیاده‌سازی تمرین، لازم است یک ویدئوی کوتاه از اجرای بخش‌های مختلف تهیه و همراه با فایل‌های تمرین تحویل داده شود.

در این ویدئو، فقط نحوه اجرای تست‌ها و نتیجه نهایی هر بخش نمایش داده شود. دانشجو نباید در ویدئو درباره جزئیات پیاده‌سازی، ساختار کد، منطق داخلی برنامه یا نحوه طراحی سیستم توضیح دهد.

هدف از این ویدئو، بررسی سریع صحت اجرای برنامه و مشاهده خروجی عملی هر بخش است. بنابراین برای هر بخش باید برنامه اجرا شود، تست مربوط به آن بخش انجام شود و خروجی قابل مشاهده نمایش داده شود.

مواردی که باید در ویدئو نمایش داده شوند:

- اجرای اسکریپت کامپایل و اجرای هر بخش
- ارسال درخواست‌های تست به برنامه
- نمایش پاسخ Master و Slave ها در بخش پایگاه داده توزیع‌شده
- نمایش تفاوت زمان پاسخ‌دهی در بخش کشینگ

- نمایش ارسال و دریافت پیام‌ها در بخش MQTT
- نمایش خواندن مقادیر سنسورها با SNMP
- نمایش خروجی API مربوط به لاگ سنسورها
- نمایش ثبت Alert در پایگاه داده در بخش Daemon

ویدئو باید کوتاه، واضح و فقط شامل تست عملکرد بخش‌ها و نمایش نتایج باشد.

موارد پژوهشی قبل از شروع پیاده‌سازی (نمره ۱۰ درصد)

قبل از شروع طراحی و پیاده‌سازی سیستم، موارد زیر را بررسی کنید و توضیح مختصری درباره هرکدام در گزارش خود بنویسید:

۱. کتابخانه Mongoose در C/C++

درباره کتابخانه Mongoose پژوهش کنید. توضیح دهید این کتابخانه چه کاربردی دارد و چگونه می‌توان از آن برای پیاده‌سازی HTTP Server و API در زبان‌های C و C++ استفاده کرد.

۲. پایگاه داده‌های توزیع‌شده

بررسی کنید چه سیستم‌هایی برای پایگاه داده توزیع‌شده توسعه داده شده‌اند. چند نمونه از این سیستم‌ها را نام ببرید و به صورت بسیار کوتاه توضیح دهید که هرکدام چه کاربردی دارند.

۳. API

درباره API پژوهش کنید و توضیح دهید API چیست، چه کاربردی دارد و چگونه باعث ارتباط بین نرم‌افزارها و سرویس‌های مختلف می‌شود.

۴. شبکه محلی یا LAN

درباره Local Area Network یا LAN پژوهش کنید و توضیح دهید شبکه محلی چیست، چه کاربردی دارد و چگونه چند نود می‌توانند در یک شبکه محلی با یکدیگر ارتباط برقرار کنند.

۵. سیستم‌های توزیع‌شده

درباره سیستم‌های توزیع‌شده پژوهش کنید و توضیح دهید این سیستم‌ها چه ویژگی‌هایی دارند، چه مزایایی ایجاد می‌کنند و چه چالش‌هایی در طراحی آن‌ها وجود دارد.

ساختار کلی پروژه باید به صورت زیر باشد:

```
project/
├── 01/
│   ├── master/
│   ├── slave/
│   ├── scripts/
│   ├── report.md
│   └── README.md
│
├── 02/
│   ├── master/
│   ├── slave/
│   ├── scripts/
│   ├── report.md
│   └── README.md
│
├── 03/
│   ├── master/
│   ├── slave/
│   ├── scripts/
│   ├── report.md
│   └── README.md
│
├── 04/
│   ├── snmp/
│   ├── scripts/
│   ├── report.md
│   └── README.md
│
├── 05/
│   ├── api/
│   ├── scripts/
│   ├── report.md
│   └── README.md
│
└── 06/
    ├── daemon/
    ├── scripts/
    ├── service/
    ├── report.md
    └── README.md
```

در صورت نیاز می‌توانید ساختار داخلی پوشه‌ها را تغییر دهید، اما هر بخش باید مستقل، قابل اجرا و دارای فایل‌های موردنیاز برای کامپایل، اجرا و تست باشد.

بخش ۱: طراحی سیستم پایه پایگاه داده توزیع شده (نمره ۳۰ درصد)

در این بخش باید سیستم پایه پایگاه داده توزیع شده را پیاده‌سازی کنید. این سیستم شامل یک نود Master و دو نود Slave است.

الزامات بخش ۱

ایجاد نودها و شبکه

- سه نود ایجاد کنید: یک نود Master و دو نود Slave
- نودها باید به صورت ماشین مجازی یا VM راه‌اندازی شوند.
- سیستم عامل برای هر نود Ubuntu 22.04 است.
- همه نودها باید در یک شبکه حضور داشته باشند و IP معتبر دریافت کنند.
- ارتباط بین نودها باید با استفاده از IP و Port انجام شود.
- هر نود باید بتواند از طریق شبکه با نودهای دیگر ارتباط برقرار کند.
- دیاگرام شبکه را طراحی کرده و در گزارش ارائه دهید.

پایگاه داده و برنامه

- هر نود باید یک پایگاه داده محلی از نوع SQLite داشته باشد.
- برای هر نود، یک اسکریپت مقداردهی اولیه پایگاه داده قرار دهید.
- برای مقداردهی اولیه پایگاه داده، از فایل‌های داده‌ای که در اختیار شما قرار داده شده است استفاده کنید.
- قبل از پیاده‌سازی، ساختار پایگاه داده و داده‌های موجود در فایل‌های اولیه را بررسی کنید و در گزارش توضیح کوتاهی درباره آن بنویسید.
- یک برنامه برای Master و یک برنامه برای Slave پیاده‌سازی کنید.
- ورودی برنامه شامل نوع سنسور و شناسه سنسور است.
- خروجی برنامه باید آخرین مقدار ثبت شده برای سنسور موردنظر باشد.
- استفاده از کتابخانه Mongoose برای پیاده‌سازی API مجاز است.
- آدرس IP و شماره Port نباید به صورت مقدار ثابت در کد نوشته شوند.
- تنظیمات ارتباطی باید از طریق فایل پیکربندی، آرگومان خط فرمان یا متغیر محیطی دریافت شوند.

رفتار مورد انتظار سیستم

- اپراتور فقط به Master درخواست ارسال می کند.
- Master ابتدا پایگاه داده محلی خود را بررسی می کند.
- اگر داده در Master موجود باشد، پاسخ به اپراتور برگردانده می شود.
- اگر داده در Master موجود نباشد، Master درخواست را به Slave اول ارسال می کند.
- اگر داده در Slave اول موجود نباشد، درخواست به Slave بعدی ارسال می شود.
- پس از پیدا شدن داده، پاسخ باید به Master برگردد و Master پاسخ نهایی را به اپراتور ارائه دهد.
- اگر داده در هیچ نودی موجود نباشد، Master باید پیام مناسب برای پیدا نشدن داده برگرداند.
- ساختار پیشنهادی فایل های بخش ۱

01/

```
|— master/
|   |— main.c یا main.cpp
|   |— Makefile
|   |— master_init_db.sh
|   |— config.example
|
|— slave/
|   |— main.c یا main.cpp
|   |— Makefile
|   |— slave_init_db.sh
|   |— config.example
|
|— scripts/
|   |— build_and_run.sh
|   |— test_requests.sh
|
|— report.md
|— README.md
```

فایل Markdown بخش ۱ باید شامل موارد زیر باشد:

۱. نحوه نصب وابستگی ها
۲. نحوه کامپایل برنامه Slave و Master
۳. نحوه اجرای Slave و Master
۴. نحوه ارسال درخواست به Master ، برای مثال با استفاده از curl
۵. توضیح ساختار پایگاه داده
۶. دیاگرام شبکه نودها
۷. مسیر درخواست و پاسخ بین Slave و Master ها
۸. بررسی امنیت پروتکل HTTP و روش های بهبود آن

امتیازدهی بخش ۱

- صحت اجرای Slave و Master
- پاسخ دهی درست به درخواست ها
- اجرای درست اسکریپت های راه اندازی و کامپایل
- استفاده درست از SQLite
- ارائه دیاگرام شبکه و توضیح مسیر ارتباطی
- مدیریت IP و Port به صورت دینامیک
- توضیح امنیت ارتباطات و روش های بهبود آن

چالش پیشرفته بخش ۱، اختیاری

در حالت پیشرفته، Master بتواند به جای استفاده از IP های متفاوت، از یک IP مشترک و Port های متفاوت برای مدیریت نودها استفاده کند.

در صورت پیاده سازی این بخش، باید موارد زیر در گزارش ارائه شود:

- دیاگرام روش پیشنهادی
- توضیح نحوه پیکربندی IP و Port ها
- نحوه اجرای برنامه ها
- اسکریپت کامپایل و اجرا
- توضیح کامل در فایل Markdown

بخش ۲: کشینگ و ساختار دو لایه پایگاه داده (نمره ۲۰ درصد)

پس از پیاده‌سازی سیستم پایه پایگاه داده توزیع‌شده در بخش اول، در این بخش باید عملکرد سیستم در خواندن داده‌های سنسورها بهبود داده شود.

برای این منظور، در هر نود یک لایه کش با استفاده از Memcached اضافه می‌شود تا سرعت پاسخ‌دهی افزایش یابد و فشار روی پایگاه داده اصلی کاهش پیدا کند.

الزامات بخش ۲

۱. ساختار دو لایه پایگاه داده

- هر نود باید یک پایگاه داده اصلی از نوع SQLite داشته باشد.
- هر نود باید یک لایه کش در حافظه RAM با استفاده از Memcached داشته باشد.
- ساختار خواندن داده باید به گونه‌ای باشد که ابتدا کش بررسی شود.
- اگر داده در کش موجود بود، پاسخ از کش برگردانده شود.
- اگر داده در کش موجود نبود، داده از SQLite خوانده شود و سپس در کش ذخیره شود.
- ساختار خواندن و نوشتن روی SQLite و Memcached باید در گزارش توضیح داده شود.

برنامه Slave و Master

- برنامه Master و Slave باید قابلیت استفاده از کش را داشته باشند.
- خروجی برنامه باید شامل آخرین مقدار سنسور و زمان پاسخ‌دهی باشد.
- برای هر برنامه باید Makefile قرار داده شود.
- برای اجرای برنامه و راه‌اندازی سرویس‌های موردنیاز باید اسکریپت Bash قرار داده شود.

ساختار پیشنهادی فایل‌های بخش ۲

02/

```
|— master/
|   |— main.c یا main.cpp
|   |— Makefile
|   |— db_init_master.sh
|   |— memcached_init_master.sh
|   |— config.example
|
|— slave/
|   |— main.c یا main.cpp
|   |— Makefile
|   |— db_init_slave.sh
|   |— memcached_init_slave.sh
|   |— config.example
|
|— scripts/
|   |— build_and_run.sh
|   |— benchmark_read.sh
|
|— report.md
|— README.md
```

اسکرپت تست سرعت خواندن سنسورها

یک اسکرپت تست بنویسید که تمام سنسورها را در دو دور بخواند:

- دور اول: داده‌ها ممکن است در کش موجود نباشند و از پایگاه داده اصلی خوانده شوند.
- دور دوم: داده‌ها باید از کش خوانده شوند، مگر اینکه طراحی سیستم شما رفتار دیگری داشته باشد.
- در گزارش باید توضیح دهید:

- کش چگونه مقداردهی می‌شود.
- چه زمانی داده از کش خوانده می‌شود.
- چه زمانی داده از SQLite خوانده می‌شود.
- اگر داده در دور دوم از کش خوانده نشد، دلیل آن چیست.
- زمان خواندن در دور اول و دوم چقدر بوده است.
- مقایسه و تحلیل زمان پاسخ‌دهی در دو حالت انجام شود.

۴. استفاده و تست

در فایل Markdown بخش ۲ باید موارد زیر نوشته شود:

- نحوه نصب Memcached
- نحوه اجرای Memcached
- نحوه کامپایل برنامه‌ها
- نحوه اجرای Master و Slave
- نحوه ارسال درخواست
- نحوه اجرای اسکریپت تست سرعت
- تحلیل نتیجه تست

بخش ۳: اتصال سیستم به پروتکل MQTT (نمره ۱۵ درصد)

در این بخش باید سیستم پیاده‌سازی شده در بخش‌های قبل را به پروتکل MQTT متصل کنید تا امکان خواندن داده‌های سنسورها از طریق MQTT فراهم شود.

در این تمرین، استفاده از MQTT نسخه ۳.۱.۱ یا نسخه ۵ مجاز است. نسخه استفاده‌شده باید در گزارش ذکر شود. همچنین سطح QoS مورد استفاده باید مشخص و دلیل انتخاب آن توضیح داده شود.

الزامات بخش ۳

- API پیاده‌سازی شده در بخش قبل باید به MQTT متصل شود.
- یک Broker برای MQTT راه‌اندازی شود. استفاده از Mosquitto مجاز است.
- Topic‌های مناسب برای درخواست و پاسخ سنسورها طراحی شود.
- ساختار Topic‌ها باید در گزارش توضیح داده شود.
- نحوه ارسال درخواست از طریق MQTT باید مشخص باشد.
- نحوه دریافت پاسخ از طریق MQTT باید مشخص باشد.
- یک اسکریپت برای فراخوانی سنسورها از طریق MQTT نوشته شود.
- اسکریپت باید سنسورها را در دو دور فراخوانی کند.

- زمان پاسخ‌دهی در هر دور اندازه‌گیری شود.
- نتایج دو دور مقایسه و تحلیل شود.
- موارد تحویلی بخش ۳
- کدهای مربوط به اتصال MQTT
- فایل Makefile
- اسکریپت کامپایل و اجرا
- فایل پیکربندی MQTT
- اسکریپت تست MQTT
- فایل Markdown شامل نحوه نصب، پیکربندی و اجرا
- گزارش شامل دیاگرام ارتباط API ، Broker ، Master و Slave ها

فایل Markdown بخش ۳ باید شامل موارد زیر باشد:

۱. نحوه نصب Broker
۲. نحوه پیکربندی MQTT
۳. ساختار Topic ها
۴. نحوه اجرای برنامه‌ها
۵. نحوه ارسال درخواست با ابزارهایی مانند mosquitto_pub
۶. نحوه دریافت پاسخ با ابزارهایی مانند mosquitto_sub
۷. نحوه اجرای اسکریپت تست
۸. مقایسه زمان پاسخ‌دهی

بخش ۴: خواندن اطلاعات سنسورها با پروتکل SNMP (نمره ۱۵ درصد)

در این بخش باید از پروتکل SNMP برای خواندن اطلاعات سنسورها استفاده کنید. لازم است ابتدا این پروتکل را بشناسید و سپس یک گزارش کوتاه درباره نحوه عملکرد آن بنویسید.

در این بخش، استفاده از snmpd پیشنهاد می‌شود.

الزامات بخش ۴

- درباره SNMP و کاربرد آن در مانیتورینگ سیستم‌ها توضیح دهید.
- از snmpd برای ارائه اطلاعات سنسورها استفاده کنید.

- برای سنسورها OID اختصاصی تعریف کنید.
- برای هر سنسور حداقل اطلاعات زیر از طریق OID قابل خواندن باشد:
- نام سنسور
- توضیحات سنسور
- آخرین مقدار ذخیره شده سنسور
- برای خواندن مقادیر، از ابزارهایی مانند snmpwalk استفاده کنید.
- یک اسکریپت اجرا قرار دهید که تمام سنسورها را با استفاده از SNMP بخواند.
- تنظیمات snmpd در فایل Markdown توضیح داده شود.
- برای ساخت OID های سفارشی، از قابلیت pass در پیکربندی snmpd استفاده کنید.

موارد تحویلی بخش ۴

- فایل های پیکربندی SNMP
- اسکریپت مربوط به OID های سفارشی
- اسکریپت خواندن همه سنسورها
- فایل Makefile در صورت نیاز به کامپایل کد
- اسکریپت کامپایل و اجرا
- فایل Markdown شامل نحوه نصب، پیکربندی و تست
- گزارش کوتاه درباره SNMP ، OID ، MIB و روش خواندن داده ها

فایل Markdown بخش ۴ باید شامل موارد زیر باشد:

۱. نحوه نصب بسته های SNMP
۲. نحوه پیکربندی snmpd
۳. ساختار OID های تعریف شده
۴. نحوه اجرای سرویس SNMP
۵. نحوه خواندن اطلاعات با snmpwalk
۶. نحوه اجرای اسکریپت تست
۷. توضیح مسیر دریافت داده از پایگاه داده تا خروجی SNMP

بخش ۵: توسعه API برای خواندن لاگ سنسورها (نمره ۵ درصد)

در این بخش باید با استفاده از کتابخانه Mongoose یک API توسعه دهید که بتواند مقادیرهای ثبت شده یک سنسور را در یک تاریخ مشخص برگرداند.

الزامات بخش ۵

- API باید با استفاده از Mongoose و در زبان C یا C++ پیاده سازی شود.
- API باید نام سنسور، شناسه سنسور و تاریخ موردنظر را در ورودی دریافت کند.
- API باید مقادیرهای ثبت شده آن سنسور را در تاریخ وارد شده برگرداند.
- داده ها باید از پایگاه داده خوانده شوند.
- پاسخ API باید ساختار مشخص و قابل خواندن داشته باشد.
- در صورت نبود داده برای تاریخ موردنظر، پیام مناسب برگردانده شود.
- برای کامپایل کد باید Makefile قرار داده شود.
- برای کامپایل و اجرای برنامه باید اسکریپت Bash قرار داده شود.
- نحوه اجرا و تست برنامه باید در فایل Markdown توضیح داده شود.

نمونه ورودی API

sensor_name=temperature

sensor_id=101

date=2026-06-01

نمونه خروجی API

```
{
  "sensor_name": "temperature",
  "sensor_id": "101",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:15:00",
      "value": "24.8"
    },
    {
```

```
"time": "10:30:00",  
"value": "25.1"  
}  
]  
}
```

موارد تحویلی بخش ۵

- کد API
- فایل Makefile
- اسکریپت مقداردهی اولیه پایگاه داده در صورت نیاز
- اسکریپت کامپایل و اجرا
- فایل Markdown شامل نحوه اجرا و تست
- گزارش شامل توضیح طراحی API، ساختار درخواست، ساختار پاسخ و مسیر خواندن داده از پایگاه داده

بخش ۶: طراحی و پیاده‌سازی سیستم Alert (نمره ۵ درصد)

در این بخش باید یک Daemon برای سیستم هشدار طراحی و پیاده‌سازی کنید. این Daemon باید مقدارهای سنسورها را پایش کند و در صورت وقوع شرایط هشدار، رویداد هشدار را در پایگاه داده ثبت کند.

الزامات بخش ۶

- یک برنامه Daemon برای پایش مقدار سنسورها نوشته شود.
- Daemon باید به صورت دوره‌ای مقدار سنسورها را بررسی کند.
- شرط‌های ایجاد هشدار باید مشخص باشند.
- هشدارها باید در یک پایگاه داده ثبت شوند.
- برای هر هشدار، حداقل اطلاعات زیر ذخیره شود:

- شناسه سنسور
- نام یا نوع سنسور
- مقدار ثبت شده
- نوع هشدار
- زمان وقوع هشدار
- وضعیت هشدار

- برای اجرای Daemon باید فایل service مربوط به systemd قرار داده شود.
- برای کامپایل برنامه باید Makefile قرار داده شود.
- برای کامپایل و اجرای برنامه باید اسکریپت Bash قرار داده شود.
- نحوه نصب، اجرا، توقف و بررسی وضعیت Daemon باید در فایل Markdown توضیح داده شود.
- نمونه شرط‌های هشدار
- دمای بیشتر از حد مجاز
- رطوبت کمتر یا بیشتر از محدوده مجاز
- عدم دریافت داده از سنسور در یک بازه زمانی مشخص
- ثبت مقدار نامعتبر برای سنسور

ساختار پیشنهادی جدول Alert

```
alerts(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  sensor_id TEXT,
  sensor_name TEXT,
  alert_type TEXT,
  sensor_value TEXT,
  created_at TEXT,
  status TEXT
)
```

موارد تحویلی بخش ۶

- کد Daemon
- فایل Makefile
- فایل service برای systemd
- اسکریپت نصب و اجرای Daemon
- اسکریپت مقداردهی اولیه پایگاه داده Alert
- فایل Markdown شامل نحوه اجرا و استفاده
- گزارش شامل دیاگرام عملکرد Daemon، منطق تولید هشدار و ساختار ذخیره‌سازی هشدارها

فایل Markdown بخش ۶ باید شامل موارد زیر باشد:

۱. نحوه کامپایل Daemon

۲. نحوه نصب فایل service

۳. نحوه اجرای Daemon

۴. نحوه توقف Daemon

۵. نحوه مشاهده وضعیت Daemon

۶. نحوه مشاهده لاگ‌های سیستم

۷. نحوه بررسی هشدارهای ثبت شده در پایگاه داده

۸. توضیح شرط‌های تولید هشدار

نکات مهم درباره گزارش‌ها

برای هر بخش، گزارش باید از دیدگاه طراحی سیستم نوشته شود، نه از دیدگاه توضیح کد. در گزارش‌ها باید به موارد زیر توجه شود:

- معماری کلی سیستم
- اجزای اصلی سیستم
- ارتباط بین اجزا
- مسیر درخواست و پاسخ
- نحوه ذخیره‌سازی داده‌ها
- نحوه مدیریت خطاها
- نحوه مدیریت تنظیمات IP و Port
- تحلیل امنیتی در بخش‌های مرتبط
- تحلیل عملکرد و زمان پاسخ‌دهی در بخش‌های کش، MQTT و تست‌ها
- دیاگرام معماری یا دیاگرام جریان داده

چک‌لیست نهایی تحویل تمرین

قبل از تحویل تمرین، موارد زیر را بررسی کنید:

- هر شش بخش در پوشه‌های جداگانه قرار گرفته باشند.
- هر بخش دارای فایل README یا Markdown باشد.
- هر بخش دارای Makefile باشد.
- هر بخش دارای اسکریپت Bash برای کامپایل و اجرا باشد.
- هیچ مقدار IP ، Port یا مسیر فایل به صورت ثابت در کد نوشته نشده باشد.

- پایگاه داده‌ها با اسکریپت‌های ارائه‌شده مقداردهی اولیه شوند.
- برنامه‌ها با فایل‌های داده تست جدید نیز قابل اجرا باشند.
- دیاگرام‌های موردنیاز در گزارش‌ها قرار گرفته باشند.
- نحوه اجرای همه بخش‌ها به‌صورت شفاف توضیح داده شده باشد.
- اسکریپت‌های تست برای بخش‌های موردنیاز قرار داده شده باشند.
- خروجی تست‌ها و تحلیل نتایج در گزارش نوشته شده باشد.
- همه کدها فقط با زبان C یا C++ نوشته شده باشند.
- پروژه ساختار منظم و قابل تصحیح داشته باشد.